

1. What is an operating system, and what are its main responsibilities?

- **An OS is the bridge between applications and computer hardware.**
- **It provides abstractions such as processes, files and virtual memory.**
- **It allocates CPU time, RAM, storage and devices among programs.**
- **It provides protection so programs cannot freely damage one another.**
- **Applications request protected services through system calls.**

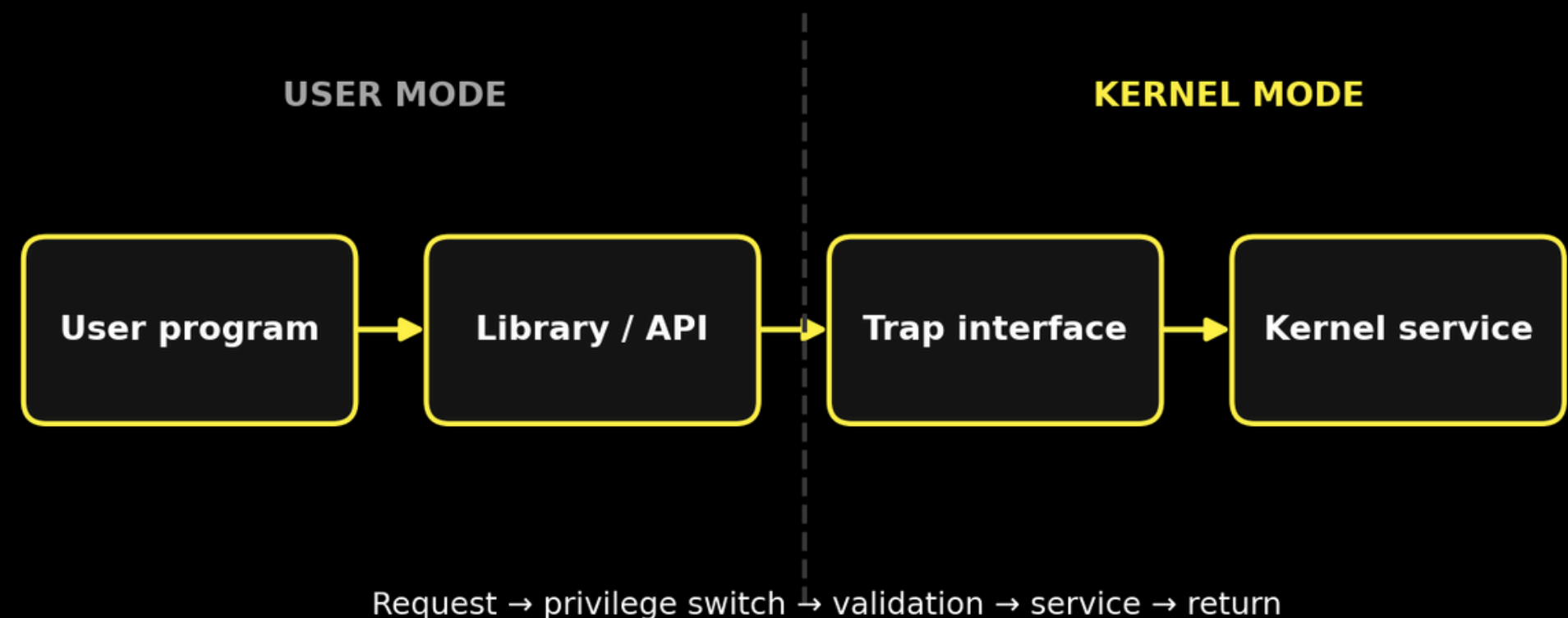
2. What is the difference between user mode and kernel mode?

Basis	User mode	Kernel mode
Purpose	Runs normal applications	Runs core OS services
Privilege	Restricted permissions	Full privileged access
Hardware	No direct hardware control	Can control devices and page tables
Entry	Normal application execution	Entered through traps, interrupts or exceptions
Failure impact	Usually affects one process	May crash or compromise the whole system

3. What is a system call, and what happens when one is executed?

- A system call lets a user program request a kernel service.
- Examples include open, read, fork and memory allocation operations.
- A library wrapper prepares the system-call number and arguments.
- A trap switches the CPU to kernel mode for validation and execution.
- The kernel returns a result or error and resumes user mode.

SYSTEM CALL FLOW



4. What is the difference between a program and a process?

Basis	Program	Process
Meaning	Passive set of instructions	Program currently in execution
Location	Stored as a file	Loaded and managed in memory
State	Has no execution state	Has registers, stack, heap and state
Instances	One executable definition	Many processes can run the same program
Example	Text-editor executable	One running text-editor instance

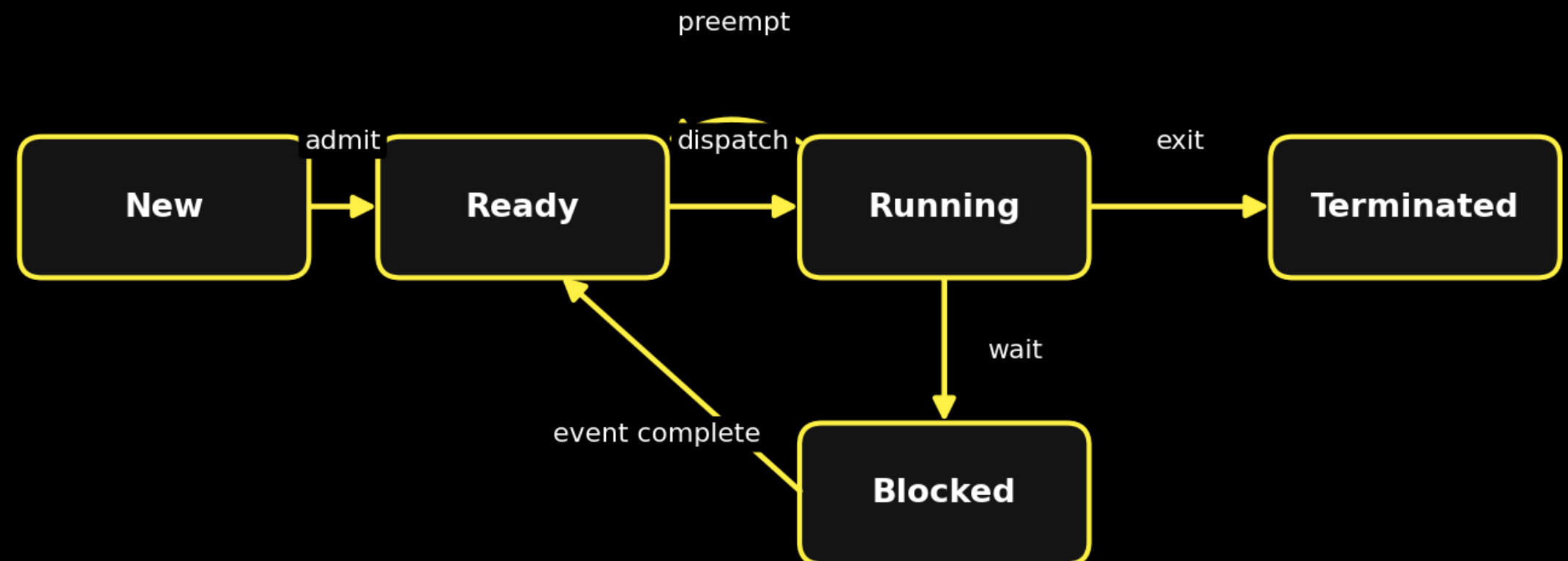
5. What is the difference between a process and a thread?

Basis	Process	Thread
Role	Resource and protection container	Execution path inside a process
Memory	Normally has a separate address space	Shares process code, heap and files
Private state	Own complete process state	Own registers, program counter and stack
Cost	Heavier to create and switch	Lighter to create and switch
Isolation	Strong fault isolation	A faulty thread can damage the process

6. What are the common states of a process?

- New means the process is being created.
- Ready means it can run but is waiting for CPU time.
- Running means its instructions are currently executing.
- Blocked means it is waiting for I/O, a lock or an event.
- Terminated means it has completed or has been stopped.

FIVE-STATE PROCESS MODEL



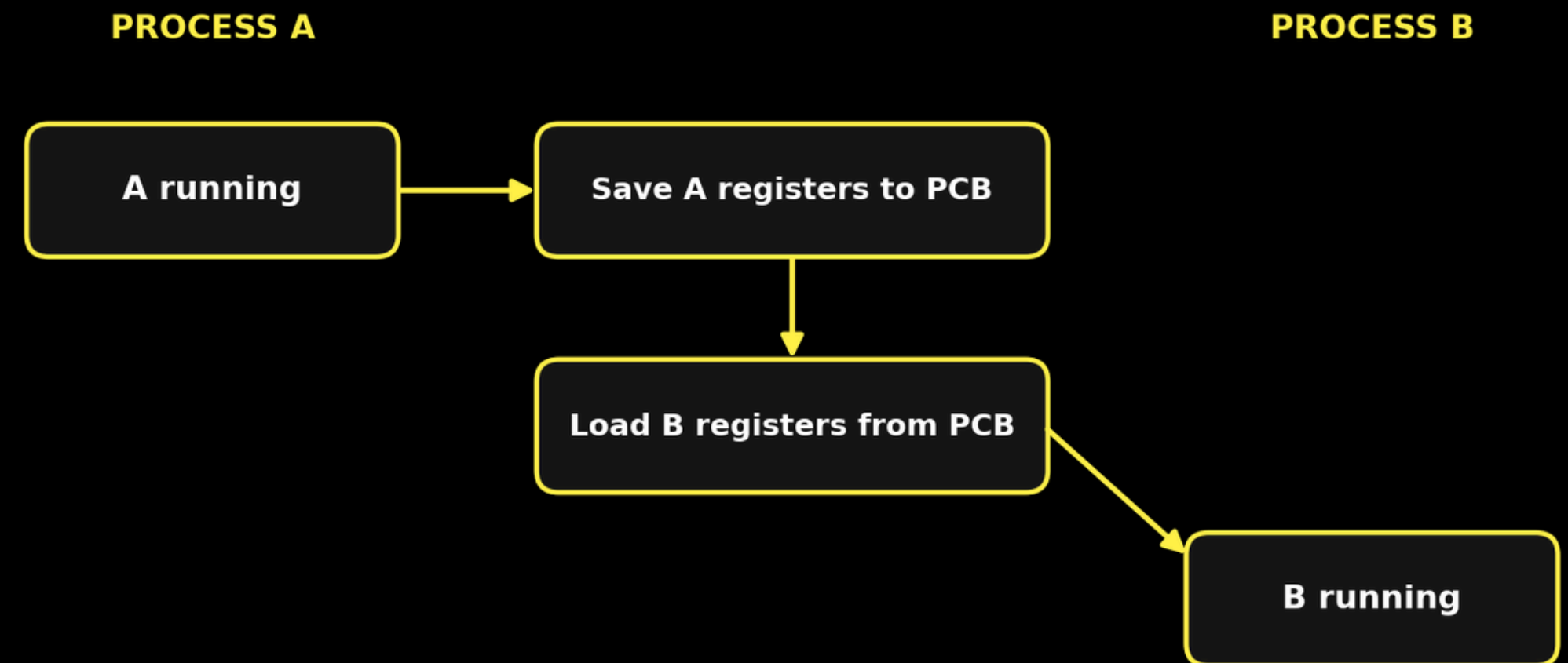
7. What is a Process Control Block (PCB)?

- **A PCB is the kernel record representing one process.**
- **It stores the process ID, state and scheduling information.**
- **It saves the program counter, registers and stack pointer.**
- **It references memory mappings, open files and security credentials.**
- **The OS uses the PCB to pause and later resume execution.**

8. What is a context switch, and why is it considered overhead?

- A context switch changes the CPU from one task to another.
- The OS saves the outgoing task's registers and execution state.
- It restores the incoming task's previously saved state.
- No application work is completed during the switch itself.
- Frequent switches also cause cache and TLB performance losses.

CONTEXT SWITCH



Useful work pauses while state, caches and address-space context change.

9. What is the difference between preemptive and non-preemptive scheduling? Which algorithms are common?

Basis	Preemptive	Non-preemptive
CPU control	OS can interrupt a running task	Task runs until it blocks or finishes
Response	Better interactive responsiveness	Slower response for waiting tasks
Overhead	More switches and synchronization	Lower switching complexity
Examples	Round Robin, SRTF, preemptive Priority	FCFS, non-preemptive SJF
Best fit	Interactive and time-sharing systems	Simple batch-style workloads

10. What is the difference between fork() and exec()?

Basis	fork()	exec()
Purpose	Creates a child process	Loads a new program
Process count	Increases by one	Does not create another process
Memory	Initially shared using copy-on-write	Replaces the current process image
Process ID	Child receives a new PID	Normally keeps the same PID
Shell use	Shell creates a child	Child runs the requested command

11. What are zombie and orphan processes?

Basis	Zombie	Orphan
State	Already terminated	Still running
Cause	Parent has not collected exit status	Original parent terminated first
CPU use	Uses no CPU	May continue using CPU and resources
OS record	Keeps a small process-table entry	Has a normal active process entry
Handling	Parent calls wait() to reap it	OS reparents it to a system process

12. What is the difference between concurrency and parallelism?

Basis	Concurrency	Parallelism
Meaning	Tasks make overlapping progress	Tasks execute at the same instant
Hardware	Possible on one CPU core	Normally needs multiple cores or units
Execution	Tasks are interleaved	Tasks run simultaneously
Main goal	Responsiveness and task coordination	Higher computational throughput
Example	One core handling many requests	Several cores processing image parts

13. What is a race condition, and what is a critical section?

- A race condition makes the result depend on execution timing.
- It usually involves unsynchronized access to shared mutable data.
- A critical section is the code accessing that shared resource.
- Only conflicting operations should enter the section together.
- Mutexes, semaphores and atomic operations can protect it.

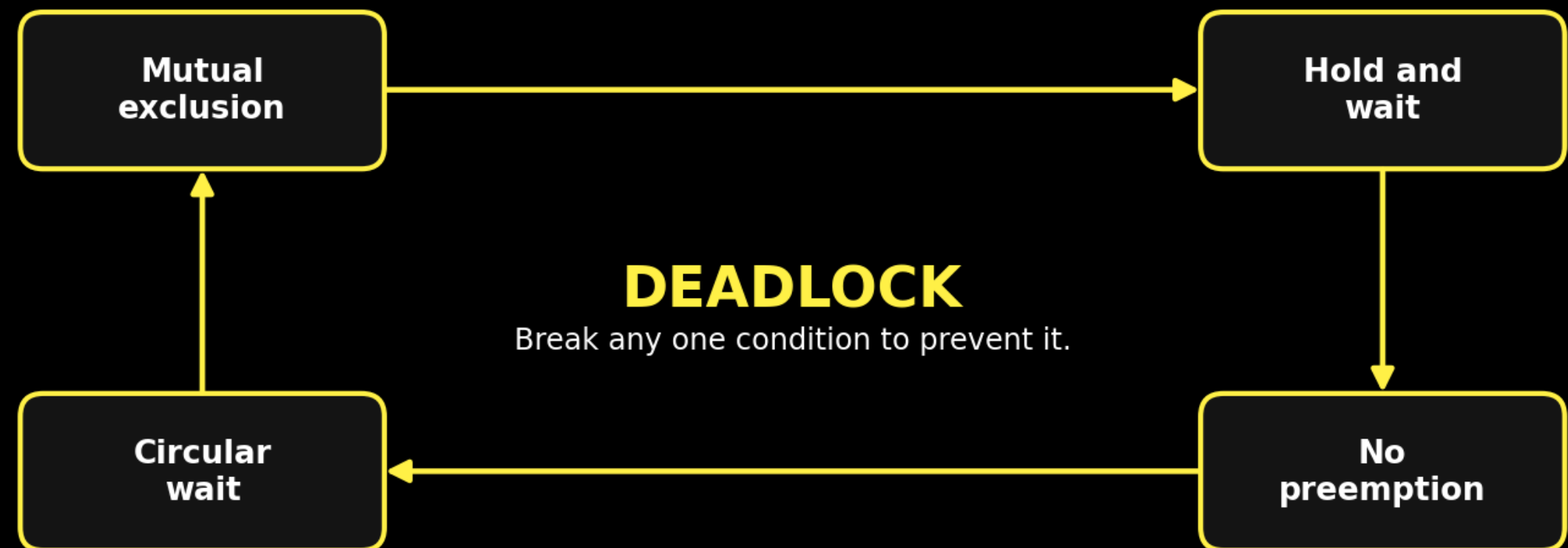
14. What is the difference between a mutex and a semaphore?

Basis	Mutex	Semaphore
Concept	Ownership-based lock	Atomic counter or signal
Ownership	Locker should unlock it	No strict ownership requirement
Value	Normally locked or unlocked	Can represent one or many resources
Main use	Protect shared critical data	Limit resources or notify availability
Release	Usually released by the owner	Another thread may signal it

15. What is deadlock, and what four conditions are required for it?

- Deadlock means a group waits forever and cannot make progress.
- Mutual exclusion requires at least one non-shareable resource.
- Hold and wait means a task holds one resource while requesting another.
- No preemption means resources are released voluntarily.
- Circular wait means the dependency chain forms a cycle.

DEADLOCK: ALL FOUR CONDITIONS TOGETHER



16. How can an operating system handle deadlocks?

- **Prevention breaks at least one required deadlock condition.**
- **Avoidance grants requests only when the state remains safe.**
- **Banker's algorithm is the textbook deadlock-avoidance method.**
- **Detection allows deadlock, then searches for blocked cycles.**
- **Recovery may terminate tasks, roll back work or preempt resources.**

17. What is the difference between deadlock, starvation and livelock?

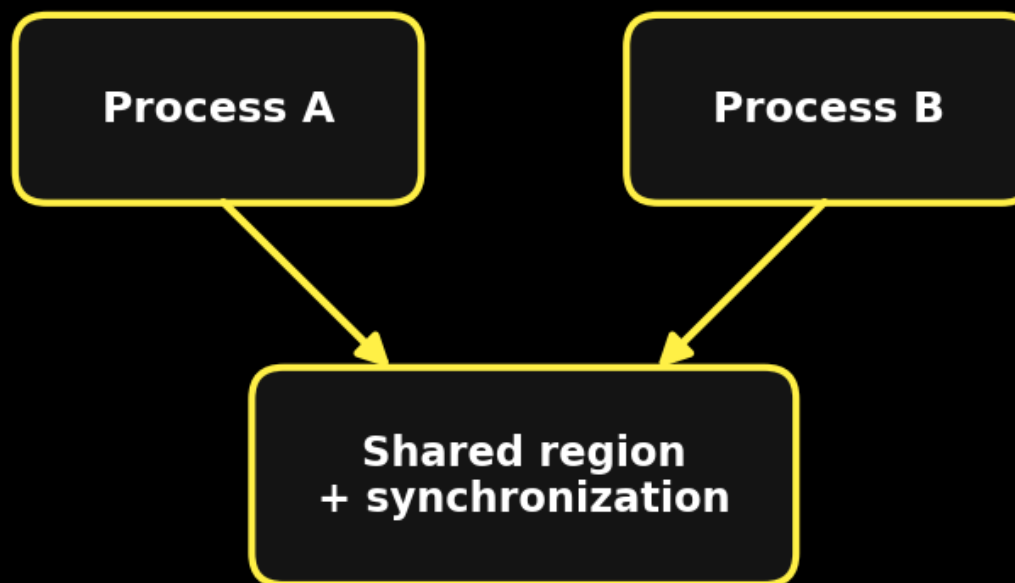
Basis	Deadlock	Starvation	Livelock
State	Participants are blocked	One task is repeatedly denied	Participants remain active
Progress	No progress for the group	Other tasks may progress	No useful progress
Cause	Cyclic resource dependency	Unfair scheduling or allocation	Repeated reactions to each other
Typical fix	Lock ordering or detection	Aging and fair scheduling	Backoff or randomized retry
Memory line	No movement	Unfair progress	Useless movement

18. What is inter-process communication (IPC), and which mechanisms are commonly used?

- IPC lets separate processes exchange data and coordinate.
- Common mechanisms include pipes, queues, shared memory and sockets.
- Shared memory is fast for large data but needs synchronization.
- Message passing gives simpler ownership and stronger separation.
- Choose using data size, latency, isolation and local-versus-network scope.

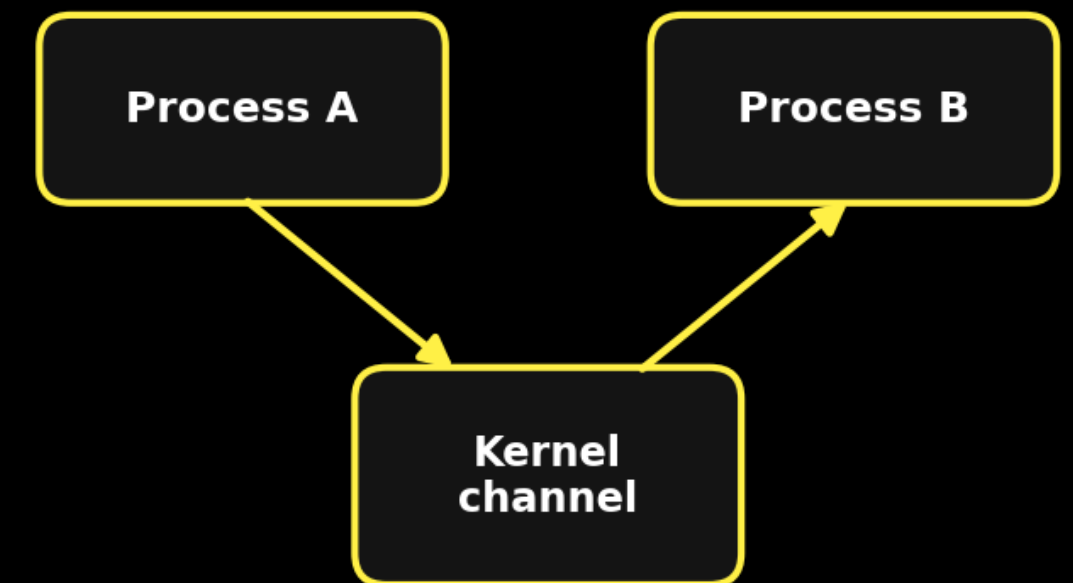
TWO COMMON IPC MODELS

SHARED MEMORY



Fast, but needs locking

MESSAGE PASSING



Simple isolation, more mediation

19. What is the difference between paging and segmentation?

Basis	Paging	Segmentation
Division	Fixed-size pages and frames	Variable-size logical regions
Program view	Mostly transparent to the programmer	Matches code, stack and data regions
Placement	Any page fits any free frame	Each segment needs suitable contiguous space
Fragmentation	May cause internal fragmentation	May cause external fragmentation
Modern use	Primary memory-management method	Limited or combined architectural use

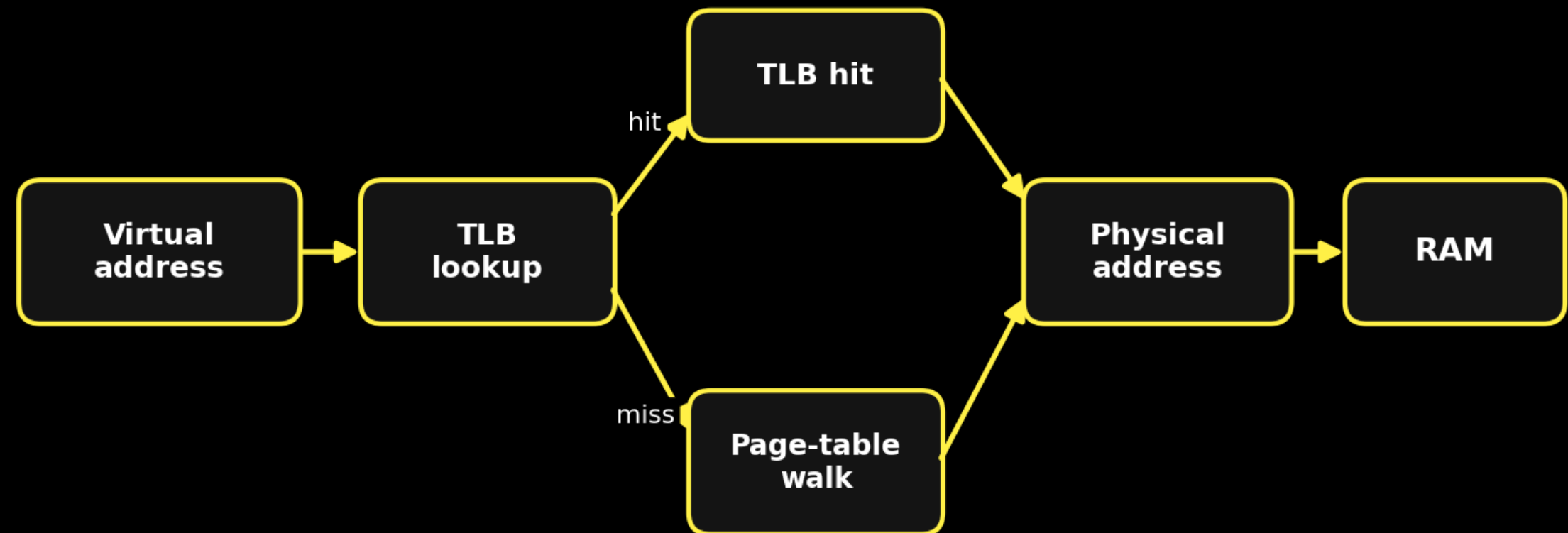
20. What is virtual memory, and what is demand paging?

- Virtual memory gives every process a private logical address space.
- Page tables map virtual pages to physical frames.
- It provides relocation, isolation and controlled memory sharing.
- Demand paging loads a page only when it is first accessed.
- A missing required page causes a page fault.

21. What is a TLB, and why is it important?

- A TLB is a small hardware cache for address translations.
- It stores recent virtual-page to physical-frame mappings.
- A TLB hit avoids an expensive page-table walk.
- A TLB miss may still find the page present in RAM.
- Therefore, a TLB miss is not automatically a page fault.

TLB ADDRESS TRANSLATION

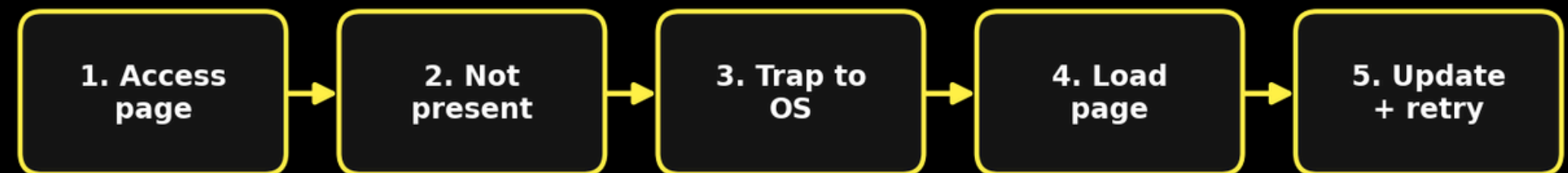


The TLB caches recent virtual-page → physical-frame mappings.

22. What is a page fault, and how does the OS handle it?

- A page fault occurs when the current mapping cannot complete an access.
- The CPU traps to the kernel for validation.
- An invalid address causes a signal or process termination.
- A valid missing page is loaded into a free or replaced frame.
- The OS updates the mapping and restarts the faulting instruction.

PAGE-FAULT HANDLING



Valid reference: obtain frame → read page → update page table/TLB

Invalid address: terminate or signal the process

23. Which page-replacement algorithms should you know, and what is Belady's anomaly?

- **Optimal replaces the page used farthest in the future.**
- **FIFO replaces the oldest loaded page and is simple.**
- **LRU replaces the least recently used page using locality.**
- **Clock or second-chance is a practical LRU approximation.**
- **Belady's anomaly means FIFO can fault more with extra frames.**

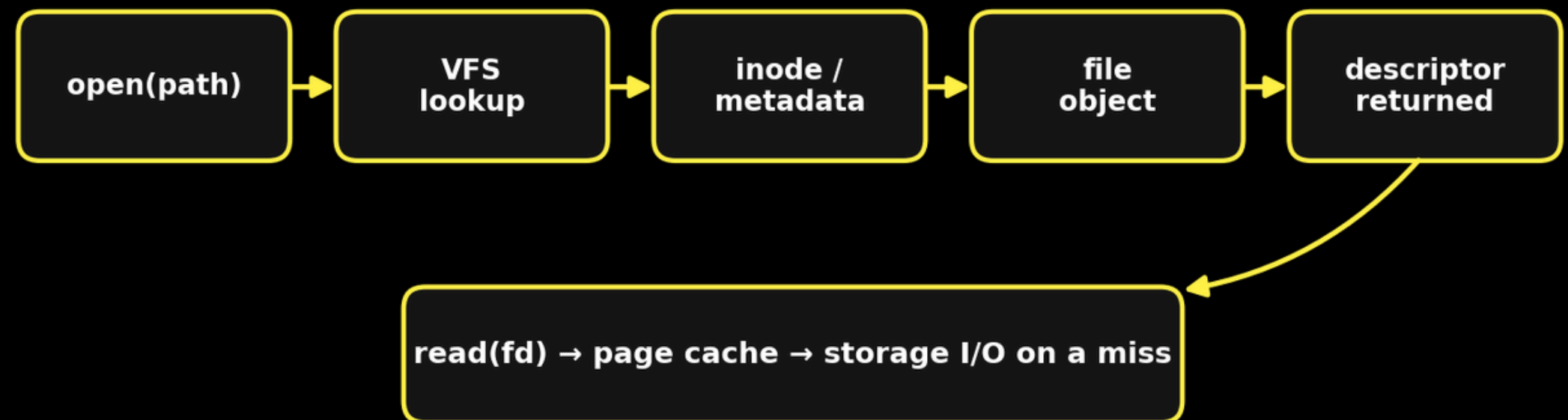
24. What is thrashing, and how can it be controlled?

- Thrashing means page faults dominate useful CPU execution.
- It occurs when active working sets do not fit in RAM.
- CPU utilization may fall while storage activity becomes very high.
- The OS can reduce multiprogramming or allocate more frames.
- Working-set and page-fault-frequency methods help control it.

25. What happens inside the OS when a program opens and reads a file?

- `open()` sends the file path and requested mode to the kernel.
- The VFS resolves the path and checks metadata and permissions.
- The kernel creates an open-file object and returns a descriptor.
- `read()` first checks whether the requested data is cached.
- On a cache miss, the file system requests storage I/O.

FROM FILE PATH TO DATA



26. What is fragmentation? Explain internal and external fragmentation.

Basis	Internal fragmentation	External fragmentation
Location	Waste inside an allocated block	Waste between allocated blocks
Cause	Allocated block is larger than requested	Free space is split into small holes
Free memory	Reserved but unused	Free but not suitably contiguous
Common with	Fixed-size allocation and paging	Variable-size contiguous allocation
Reduction	Use better size classes or smaller blocks	Coalesce holes or compact memory

Internal



External



27. What are the main types of kernels?

Monolithic

Most services
in kernel space

Microkernel

Minimal kernel;
services in user space

Hybrid

Combines both
major approaches

**Modular /
Exokernel**

Loadable parts or
minimal abstractions

More services together

More separation / specialization

28. What is the difference between multiprogramming, multitasking and multiprocessing?

Basis	Multiprogramming	Multitasking	Multiprocessing
Meaning	Several programs remain in memory	Tasks share CPU through time slices	Several processors or cores execute work
Switch trigger	Usually switches when a task waits	Switches rapidly using preemption	Work runs on different units
Parallelism	Not required	Not required	Supports true parallel execution
Main goal	Keep the CPU busy	Improve interactive responsiveness	Increase speed and throughput
Modern systems	Used	Used	Used together with both techniques

29. What is the difference between an interrupt and an exception?

Basis	Interrupt	Exception
Source	Usually external hardware	Current instruction or software
Timing	Asynchronous	Synchronous
Examples	Timer, keyboard or device completion	Page fault, divide-by-zero or trap
Instruction link	Independent of the current instruction	Directly related to current execution
Handling	OS runs an interrupt handler	OS handles, signals, restarts or terminates

30. What is copy-on-write, and why is it useful?

- Copy-on-write lets processes initially share the same physical pages.
- The shared pages are marked read-only in their mappings.
- A write attempt triggers a protection fault.
- The OS copies only that page and updates the writer's mapping.
- It makes fork and snapshot-style operations faster and memory-efficient.